TRANSKRIP PROMPT DAN JAWABAN AI

Conditional GAN (cGAN) Fashion-MNIST — Bagian 5, 6, dan 7

Dokumen ini menyusun percakapan secara kronologis dengan pemisahan prompt pengguna dan jawaban AI. Ringkasan konsep, keputusan desain, hasil pengujian, evaluasi, eksperimen ablation, dan kesimpulan presentasi dipertahankan sesuai transkrip sumber. Catatan status operasional yang tidak berisi materi percakapan substantif dihilangkan.

Topik: Discriminator • Training Loop • Evaluasi • Hyperparameter Tuning/Ablation • Interpretasi Hasil

Prompt awal — Discriminator (Bagian 5)

PROMPT / PESAN PENGGUNA

Kami kelompok mahasiswa Informatika yang membangun Conditional GAN dari nol (from scratch) dengan PyTorch untuk Fashion-MNIST, dalam satu notebook Colab. Saya bertugas sebagai Anggota 3 (Training Loop & Hyperparameter Tuning), yaitu Bagian 5 (Discriminator), Bagian 6 (Training loop), dan Bagian 7 (Evaluasi).

Konteks yang saya lampirkan:

1. Cetak biru dari ketua kelompok (PDF). Ikuti kontrak antar-bagian dan spesifikasinya persis.

2. Notebook dari teman saya (Bagian 3 dan 4): get_dataloader/train_loader, Classifier pendamping, dan fungsi generate(checkpoint, class_id, n). Bagian 0 sampai 2 (CFG, utilitas, Generator) ada di repo kelompok dan dijalankan lewat %run, jadi pakai nama CFG, Generator, dan fungsi utilitas yang sama.

Aturan:

- Kode hanya untuk bagian saya. Jangan menulis ulang Bagian 0 sampai 4.

- Kode modular, diberi komentar bahasa Indonesia yang menjelaskan alasan tiap bagian, dan tidak disalin dari tutorial atau repo orang lain.

- Jelaskan konsepnya dulu dengan bahasa sederhana sebelum memberi kode, supaya saya bisa mempresentasikannya.

- Kita bekerja bertahap per bagian. Setiap error akan saya kirim beserta pesan error lengkapnya.

- Jika ada yang tidak jelas atau tidak cocok dengan kode temanku, tanyakan dulu, jangan menebak.

Mulai dengan Bagian 5: Discriminator D(x [B,1,28,28], y [B]) yang menghasilkan logit [B] tanpa Sigmoid. Label diubah lewat nn.Embedding(10, 784), di-reshape ke 1x28x28, lalu di-concat ke gambar. Ikuti tabel layer di cetak biru bagian 3, inisialisasi N(0, 0,02), dan sediakan opsi dropout dan spectral_norm (default mati). Sertakan tes: bentuk keluaran, jumlah parameter (±0,44 juta), dan pasangan (gambar, label salah) harus memberi logit berbeda dari pasangan benar.

JAWABAN AI

Saya mulai dengan membaca cetak biru dan notebook temanmu, supaya kode Bagian 5 cocok persis dengan kontraknya.

Sedang membaca cetak biru dan notebook tim (CFG, weights_init, Generator, checkpoint), lalu menyusun Bagian 5. Sebentar lagi penjelasan konsep dan kodenya siap.

Kode Discriminator sudah tertulis. Sekarang saya memasang PyTorch di sandbox supaya tes bisa benar-benar dijalankan sebelum dikirim ke kamu.

Masih memasang PyTorch di sandbox (unduhan besar) untuk menguji Bagian 5 secara nyata. Kode dan tes sudah siap, tinggal dijalankan.

Masih menunggu PyTorch selesai terpasang di sandbox. Begitu siap, saya jalankan tes Bagian 5 dan kirim hasilnya.

Unduhan sedang berjalan (cuDNN sudah selesai, tinggal beberapa paket CUDA lain). Saya tunggu notifikasi.

Stabil. Kirim notebook-nya.

Bagian 5 sudah jadi dan lulus semua tes di sandbox. Tes memakai kode asli repo tim (CFG, weights_init, Generator) dan data acak sebagai pengganti train_loader, jadi belum dicoba dengan Fashion-MNIST sungguhan.

Konsep (untuk presentasi)

Discriminator adalah juri GAN. Ia menerima gambar dan labelnya, lalu mengeluarkan satu angka (logit) per gambar. Angka positif berarti “asli dan cocok dengan label”, angka negatif berarti “palsu atau tidak cocok”. Angka ini sengaja belum jadi probabilitas, karena Sigmoid sudah ada di dalam BCEWithLogitsLoss (Bagian 6).

Cara label masuk ke D:

Label 0 sampai 9 diubah nn.Embedding(10, 784) menjadi 784 angka yang dipelajari.

Angka itu dibentuk ulang menjadi peta 1×28×28.

Peta itu ditumpuk ke gambar sebagai kanal kedua, jadi [B,2,28,28].

Setiap konvolusi melihat piksel gambar dan label di posisi yang sama. D bisa menolak “gambar sepatu berlabel baju”, dan tekanan inilah yang memaksa G menaati label.

Arsitektur: tiga konvolusi stride 2 (28→14→7→4) dengan LeakyReLU 0,2, lalu Flatten dan Linear(4096,1). Layer pertama tanpa BatchNorm. Hasilnya 440.801 parameter, lebih kecil dari G supaya D tidak menang terlalu cepat.

Saklar cadangan untuk Bagian 6, default-nya mati:

Dropout: Dropout2d setelah LeakyReLU di layer 4 dan 5, sehingga D tidak selalu yakin.

Spectral norm: semua Conv dan Linear dibungkus spectral_norm dan BatchNorm dibuang, sehingga reaksi D lebih halus.

Hasil tes

Tes Hasil

Bentuk keluaran logit [B] float32, tanpa Sigmoid

Jumlah parameter 440.801, cocok dengan rumus dan target ±0,44 juta

Label salah selisih logit minimal ±1e-5 untuk semua gambar (dites 4 seed data)

Mini-training 150 langkah D menilai pasangan benar lebih tinggi dari label salah pada 100% pasangan

Dropout, spectral norm, gradien ke gambar, simpan-muat, input salah lulus

Di awal training D belum tahu label mana yang benar. Tes “label salah” hanya membuktikan logitnya berbeda, sedangkan arah selisihnya (benar lebih tinggi dari salah) dibuktikan oleh mini-training.

Yang cocok dengan kode temanmu

CFG ternyata sudah punya d_channels, leaky_slope, d_dropout, d_spectral_norm, dan d_embed_dim (=784). D membaca semuanya dari sana, jadi di Bagian 6 cukup mengubah CFG.d_dropout atau CFG.d_spectral_norm. Kelas ini memakai weights_init dan pola validasi input yang sama dengan Generator.

Tiga keputusan yang perlu kamu konfirmasi

Embedding label di D tidak diinisialisasi N(0, 0,02). weights_init milik tim sengaja membiarkan Embedding memakai default PyTorch, dan G juga begitu. Saya mengikuti konvensi itu. Kalau mau N(0, 0,02), tinggal ubah satu baris.

Di mode spectral norm, konvolusi layer 4 dan 5 diberi bias. Cetak biru memakai bias=False hanya karena ada BatchNorm sesudahnya. Kalau BatchNorm dibuang, bias-nya tetap berguna. Cetak biru tidak menyebut hal ini.

Jangan panggil D.apply(weights_init) lagi di Bagian 6. Inisialisasi sudah dilakukan di dalam konstruktor, sebelum spectral norm dipasang. Memanggilnya sesudah itu bisa error atau tidak berefek di mode spectral norm.

Jalankan sel 2 lalu sel 3. Sel 3 memakai train_loader dari Bagian 3 dan G dari Bagian 2, jadi G harus sudah dibuat. Kalau ada error, kirim pesan lengkapnya.

Kalau sudah aman, saya lanjut ke Bagian 6 (training loop).

Notifikasi itu hanya pemantau latar belakang yang kedaluwarsa. Tidak ada yang perlu dilakukan, karena PyTorch sudah terpasang dan tes sudah selesai.

Bagian 5 sudah selesai dan menunggu konfirmasimu. Notebook bagian5_discriminator.ipynb sudah terkirim. Isinya penjelasan konsep, kelas Discriminator, dan tes yang semuanya lulus di sandbox.

Sebelum lanjut ke Bagian 6 (training loop), saya butuh dua hal darimu:

Jalankan sel 2 dan sel 3 di Colab, lalu kabari hasilnya. Kalau ada error, kirim pesan lengkapnya.

Konfirmasi tiga keputusan desain di pesan sebelumnya: inisialisasi Embedding D, bias di mode spectral norm, dan tidak memanggil D.apply(weights_init) lagi di Bagian 6.

Konfirmasi Bagian 5 dan instruksi Training Loop (Bagian 6)

PROMPT / PESAN PENGGUNA

Sel 2 dan sel 3 Bagian 5 sudah saya jalankan di Colab (di salinan notebook Bianka, GPU aktif) dan hasilnya LULUS.

Ringkasan hasil:

- Discriminator: Embedding(10, 784), 3 konvolusi (2→64, 64→128, 128→256) dengan LeakyReLU 0,2, Flatten, Linear(4096, 1)

- Jumlah parameter: 440.801 (embed 7.840, features 428.864, head 4.097)

- Bentuk keluaran tiap tahap sesuai (embed 128x784, features.5 128x256x4x4, head.1 128x1)

- Selisih |logit benar - logit salah|: min 2,35e-04, rata-rata 1,01e-02

- Setelah 150 langkah mini-training: loss 0,001, pasangan benar > salah pada 100%

- TES 5 LULUS

Saya setuju dengan ketiga keputusan desain:

1. Embedding D memakai inisialisasi default PyTorch (sama seperti G)

2. Di mode spectral norm, konvolusi layer 4 dan 5 diberi bias

3. D.apply(weights_init) tidak dipanggil lagi di Bagian 6

Lanjut ke Bagian 6: training loop 50 epoch sesuai cetak biru bagian 4 dan 5. Ketentuannya: BCEWithLogitsLoss, target real 0,9, target fake 0,0, target G 1,0, Adam lr 2e-4 betas (0,5; 0,999) untuk G dan D (dua optimizer terpisah), satu langkah D lalu satu langkah G per iterasi. Di langkah D, pakai .detach() pada gambar fake dan pakai y dari batch real untuk fake. Pakai z baru dari N(0, I) tiap iterasi. L_D adalah jumlah bagian real dan fake, dan tiap suku dirata-rata atas batch. Simpan checkpoint (epoch, G, D, opt_G, opt_D, config) tiap 10 epoch dan di akhir ke Drive. Tulis log CSV per iterasi: loss G, L_D, D(real), D(fake). Simpan grid sampel 10x10 tiap 5 epoch (dan epoch 1) dengan z_fixed dan y_fixed seperti di cetak biru bagian 7, dengan G.eval() saat membuat sampel lalu G.train() lagi. Sertakan fungsi untuk melanjutkan training dari checkpoint, dan tes awal dengan 1 epoch kecil (misalnya beberapa puluh iterasi) sebelum menjalankan 50 epoch penuh. Pakai CFG, set_seed, make_fixed_noise, save_image_grid, dan fungsi checkpoint milik tim. Jelaskan konsepnya dulu dengan bahasa sederhana, beri komentar di kodenya, lalu kirim sebagai sel notebook yang bisa langsung saya tempel ke salinan notebook.

JAWABAN AI

Saya cek dulu apa saja yang dibutuhkan Bagian 6 dari kode tim, lalu menyiapkan lingkungan uji.

Bagian 6 sudah jadi dan lulus semua tes di sandbox. Tes memakai kode asli repo tim dengan data acak di CPU, jadi belum dicoba dengan Fashion-MNIST dan GPU.

Konsep (untuk presentasi)

GAN dilatih seperti permainan dua pemain: G memalsukan, D menjadi juri. Di setiap iterasi:

G membuat gambar palsu dari noise baru dan label dari batch real.

Latih D: (real, y) harus dinilai 0,9 dan (fake, y) harus dinilai 0,0. Gambar fake di-.detach() supaya langkah ini tidak ikut mengubah G.

Latih G: fake yang sama harus dinilai D sebagai 1,0. Kali ini tanpa detach, karena gradien memang harus sampai ke G.

Target real 0,9 (bukan 1,0) menjaga D agar tidak terlalu yakin. D yang terlalu yakin membuat gradien ke G mati. L_D adalah jumlah bagian real dan fake, sedangkan L_G satu suku.

Isi notebook

Empat sel, ditempel berurutan setelah Bagian 5:

Penjelasan konsep dan tabel keputusan desain.

Kode training: d_step, g_step, train_iteration, dan train, plus fungsi sampel, log, dan resume.

TES 6.

Sel run: satu baris out = train().

Yang dibuat sesuai permintaanmu:

Log CSV: satu baris per iterasi (train_iter.csv) dan satu per epoch (train_epoch.csv).

Grid sampel: epoch_XXX.png di epoch 1 lalu tiap 5 epoch, dengan noise tetap. G dalam mode eval saat membuat sampel, lalu modenya dikembalikan.

Checkpoint: tiap 10 epoch dan di epoch terakhir.

Resume: train() otomatis lanjut dari checkpoint terbaru di folder. Bagian 6 memakai CFG, set_seed, make_fixed_noise, save_image_grid, dan fungsi checkpoint milik tim.

Yang dites

Detach: langkah D tidak memberi gradien ke G dan tidak mengubah bobot G.

Pemisahan optimizer: langkah G tidak mengubah bobot D.

Rumus loss: L_D dan L_G cocok dengan hitungan manual (target 0,9, 0,0, dan 1,0).

Smoke test: 3 epoch × 20 iterasi di folder sementara, jadi Drive tidak tersentuh. Dicek jumlah baris CSV, nama file sampel dan checkpoint, dan grid epoch 1 berbeda dari epoch 2.

Resume: log tidak menggandakan epoch, dan baris sisa dari run yang terputus dibuang.

Perlindungan lain: loss NaN menghentikan training dengan pesan jelas, dan config yang tidak cocok ditolak.

Hal yang perlu kamu ketahui

Nilai acuan 1,386 dan 0,693 tidak berlaku di iterasi pertama. Di tes saya, iterasi pertama memberi L_D 1,68 dan D(real) 0,35. Penyebabnya BatchNorm di D mode train. Ini normal. Acuan itu berlaku saat seimbang dan tetap dipakai sebagai garis di grafik Bagian 7.

Run ablation wajib memakai folder sendiri. Pakai train(dirs=run_dirs("dropout")) supaya checkpoint, log, dan grid baseline tidak tertimpa. Run baru di folder yang sudah punya checkpoint ditolak.

Mengubah CFG.lr_d lalu resume sekarang berlaku. Saya menimpa lr optimizer dari CFG, karena load_state_dict membawa lr lama. Tanpa itu, resep “turunkan lr D” di bagian 9 diam-diam tidak berefek.

Urutan batch data saat resume. Loader Bianka diseed tetap, jadi setelah resume urutan batch mengulang dari epoch 1. Secara statistik ini tidak bermasalah.

Langkah berikutnya

Jalankan sel 2 dan 3 di Colab. Smoke test mencetak estimasi waktu 50 epoch di GPU-mu, yang sebaiknya mendekati 5–10 menit. Kalau tes lulus, jalankan sel 4 untuk training penuh. Kalau ada error, kirim pesan lengkapnya. Setelah training selesai dan ada log, saya lanjut ke Bagian 7 (evaluasi).

Konfirmasi training baseline dan instruksi Evaluasi (Bagian 7)

PROMPT / PESAN PENGGUNA

Bagian 6 selesai di Colab (T4). TES 6 lulus dan training baseline 50 epoch selesai tanpa error, ±18 dtk/epoch (±15 menit total). Ringkasan log epoch: epoch 1 L_G 1,806 / L_D 1,020 / D(real) 0,604 / D(fake) 0,293; epoch 10 L_G 1,472 / L_D 1,105; epoch 25 L_G 1,871 / L_D 0,913; epoch 50 L_G 2,610 / L_D 0,684 / D(real) 0,735 / D(fake) 0,164. Checkpoint ada di epoch 10, 20, 30, 40, 50. Tren: D makin unggul pelan-pelan (L_D turun, L_G naik), belum melewati ambang bahaya cetak biru bagian 9.

Lanjut Bagian 7 (evaluasi) sesuai cetak biru bagian 7: kurva loss G dan D dengan garis acuan 1,386 dan 0,693 (dari train_epoch.csv dan train_iter.csv), grafik D(real) vs D(fake), akurasi classifier pada 1.000 gambar buatan per kelas lewat generate() milik temanku (checkpoint epoch 10, 20, 30, 40, 50, target total ≥ 80%, bandingkan dengan akurasi classifier pada data asli), dan keragaman per kelas (rasio buatan/real, ambang 0,5). Semua grafik dari file log. Buat sebagai sel notebook yang bisa saya tempel, dan jelaskan cara membaca tiap grafik untuk presentasi.

JAWABAN AI

Saya baca dulu antarmuka classifier dan generate() milik temanmu supaya Bagian 7 memakainya persis.

Ran a command

Ran a command

Saya sudah paham antarmuka temanmu: load_classifier(), evaluate_classifier(), load_generator(), dan sample_from_generator(). Sekarang saya tulis Bagian 7 dan mengujinya.

Kode Bagian 7 sudah lulus semua tes di sandbox. Sekarang saya periksa tampilan grafiknya sebelum membungkus notebook.

Bagian 7 sudah jadi dan lulus semua tes di sandbox. Tes itu memakai kode asli repo tim, kode classifier dan generate milik temanmu, serta data tiruan karena sandbox tidak bisa mengunduh Fashion-MNIST. Angka akurasi dan keragaman yang sesungguhnya baru muncul saat kamu menjalankannya di Colab.

Isi notebook

Lima sel, ditempel berurutan setelah Bagian 6:

Konsep dan cara membaca grafik. Isinya penjelasan lima ukuran, lalu per grafik: tampilan sehat, tampilan bahaya, dan kalimat untuk presentasi. Keterbatasan tiap metrik juga ditulis.

Metrik dan CSV. Menghitung akurasi dan keragaman lalu menulis tiga CSV di logs/.

Grafik dan diagnosis. Enam grafik dibaca dari log dan CSV, plus diagnosis otomatis.

TES 7.

Sel run. Isinya load_classifier(), compute_real_reference(clf), dan evaluate_all(...).

Yang dihasilkan

Kurva loss G dan D. Garis tebal adalah rata-rata per epoch, garis pudar adalah rata-rata bergerak 100 iterasi. Garis acuan ada di 1,386 dan 0,693.

```
D(real) vs D(fake). Ada pita sehat (0,6–0,8 dan 0,2–0,4) dan garis bahaya 0,9 / 0,1.
```

Akurasi classifier. Per checkpoint (epoch 10 sampai 50), dibandingkan dengan target 80% dan akurasi data asli, plus per kelas.

Keragaman per kelas. Rasio buatan/real dengan ambang 0,5.

Dua grafik tambahan. Matriks kebingungan (untuk melihat kelas mana yang tertukar) dan grid sampel epoch 1/10/25/50 di samping grid acuan real.

Penyimpanan. Semua grafik disimpan sebagai PNG di folder figures/ untuk Sri.

Hal yang perlu kamu ketahui

Grafik dan angka dijamin berasal dari file. Tes mencocokkan titik di grafik dengan isi CSV. Notebook juga menolak log yang epoch-nya tidak berurutan 1..N atau yang log iterasinya tidak cocok dengan log epoch.

Seed evaluasi sengaja berbeda dari seed grid. Evaluasi memakai CFG.seed + 1, supaya G tidak diuji dengan noise yang sama persis seperti di grid.

Ambang diagnosis bukan standar baku. “Beberapa epoch” di cetak biru saya terjemahkan menjadi 3 epoch berturut-turut. Angka ini bisa diubah di EVAL_CFG.

Rasio keragaman di atas 1 tidak otomatis bagus. Gambar penuh noise juga berkeragaman tinggi.

Bacaan awal dari angka yang kamu kirim

Ini bacaan dari ringkasan log-mu, bukan dari file log-nya langsung:

L_D 0,684 ada di atas ambang 0,6, dan L_G 2,61 ada di bawah 3. Keduanya aman.

```
D(real) 0,735 ada di pita sehat.
D(fake) 0,164 sedikit di bawah pita sehat 0,2–0,4, tapi masih di atas batas bahaya 0,1. Diagnosis otomatis akan
menandainya PERHATIAN, bukan WASPADA.
```

Artinya D memang unggul pelan-pelan, tapi belum melewati ambang bahaya. Kesimpulan akhirnya tergantung dua hal yang baru ketahuan setelah sel run dijalankan: akurasi ≥ 80% dan rasio keragaman ≥ 0,5. Kalau keduanya terpenuhi, baseline sudah layak. Dropout dan spectral norm tetap bisa dicoba sebagai ablation untuk bahan presentasi, dengan train(dirs=run_dirs("dropout")) supaya baseline tidak tertimpa.

Jalankan sel 2–4, lalu sel 5. Kalau ada error, kirim pesan lengkapnya. Kirim juga tabel akurasi dan keragaman yang tercetak, supaya saya bantu menyusun interpretasi untuk presentasi.

Hasil evaluasi baseline dan permintaan eksperimen ablation

PROMPT / PESAN PENGGUNA

Bagian 7 sudah saya jalankan di Colab dan TES 7 lulus. Evaluasi baseline 50 epoch (epoch 10, 20, 30, 40, 50) hasilnya:

- Akurasi classifier pada gambar buatan: 91,75%, 92,32%, 92,30%, 92,13%, 92,33% (classifier pada data asli: 91,32%)

- Rasio keragaman terendah: 0,86, 0,83, 0,89, 0,87, 0,92 (kelas Trouser), semuanya di atas ambang 0,5

- Per kelas epoch 50: kelas terendah Shirt 80,00% (data asli 74,90%), tertinggi Trouser 99,20%

- Diagnosis otomatis: L_D 0,718 OK, L_G 2,550 OK, D terlalu yakin 0 epoch OK, akurasi OK, keragaman OK. Hanya D(real)/D(fake) = 0,725 / 0,175 berstatus PERHATIAN karena D(fake) sedikit di bawah pita sehat 0,2-0,4.

- Kurva loss: L_G naik hampir lurus dari 1,41 (epoch 8) ke 2,61 (epoch 50), L_D turun ke 0,68. Grid sampel: epoch 1 kabur, epoch 10 sudah berbentuk pakaian, epoch 25 sampai 50 makin tajam, baris sesuai kelas, kolom bervariasi.

Tolong:

1. Jelaskan dengan bahasa sederhana untuk presentasi: kenapa L_G naik dan L_D turun padahal gambar dan akurasinya bagus, dan kenapa akurasi gambar buatan (92,33%) bisa sedikit di atas data asli (91,32%).

2. Buatkan kode eksperimen tuning/ablation sesuai cetak biru bagian 6 dan 9, satu perubahan per run, tiap run di folder sendiri lewat run_dirs: (A) CFG.d_dropout = 0,3; (B) CFG.lr_d = 1e-4. Setiap run dikembalikan ke nilai baseline setelah selesai dan dievaluasi dengan evaluate_all memakai clf dan ref yang sama.

3. Buatkan satu sel untuk membandingkan baseline dengan run A dan B dalam satu tabel (akurasi total, rasio keragaman terendah, L_D dan L_G akhir, D(real) dan D(fake) akhir), plus grid epoch 50 berdampingan.

Jelaskan konsepnya dulu, beri komentar di kode, dan beri tahu perkiraan waktu per run (baseline ±18 detik per epoch di T4).

JAWABAN AI

Saya cek dulu tipe field CFG (supaya cfg_override aman), lalu menulis kode ablation.

Notebook Bagian 8 sudah jadi dan lulus TES 8 di sandbox. Tesnya memakai run mini (2 epoch) dengan data tiruan, jadi mesin eksperimennya sudah teruji tetapi angka hasil run A dan B baru muncul di Colab.

1. Penjelasan untuk presentasi

Kenapa L_G naik dan L_D turun padahal gambar dan akurasinya bagus

Loss GAN bukan “nilai ujian gambar”. Ia mengukur hasil pertandingan G melawan D, dan D ikut belajar.

Bayangkan G sebagai murid dan D sebagai penguji. Penguji makin teliti dan mengenali ciri halus yang tidak terlihat oleh mata maupun classifier. Murid makin sering ditolak (L_G naik) walaupun jawabannya makin bagus. Penguji makin sering benar (L_D turun).

Angkamu konsisten dengan itu. D(fake) turun ke 0,175, sehingga L_G = −log D(fake) membesar. Akurasi justru hampir datar sejak epoch 20, jadi kenaikan L_G sesudahnya tidak diikuti penurunan ketaatan label.

G tetap belajar karena memakai bentuk non-saturating: selama D(fake) belum mendekati 0, gradiennya masih mengalir. Batas bahaya cetak biru belum dilewati.

Yang belum terbukti: “D makin teliti” adalah tafsir standar, bukan hasil eksperimen kita. Ablation A dan B menguji tafsir itu. Kalau D diperlemah, L_G seharusnya turun dan D(fake) naik.

Alasan tetap waspada: L_G naik hampir lurus, jadi bila training diperpanjang bisa menuju zona bahaya.

Kenapa akurasi buatan (92,33%) sedikit di atas data asli (91,32%)

Classifier hanya mengukur apakah label ditaati dan gambarnya mudah dikenali. Ia tidak mengukur apakah gambar buatan lebih bagus dari yang asli.

Dugaan utama: gambar G cenderung prototipikal, yaitu mengumpul di bentuk khas tiap kelas. Test set asli memuat gambar ambigu, misalnya kemeja yang mirip T-shirt.

Dugaan ini didukung datamu: kelas Shirt punya selisih terbesar (asli 74,9%, buatan 80,0%).

Faktor kedua: D menerima label sebagai kanal tambahan dan menolak pasangan yang tidak cocok, sehingga G menonjolkan ciri pembeda kelas.

Selisihnya kecil, 1,0 poin persentase dengan 10.000 gambar per sisi. Galat baku per sisi ±0,27 poin, jadi kemungkinan nyata tetapi jangan diklaim berlebihan.

Dugaan prototipikal bisa dicek lewat sel 8.6, yang membandingkan keyakinan classifier pada gambar real dan buatan. Kalau gambar buatan lebih diyakini, dugaan itu terdukung.

2. Eksperimen A dan B

Baseline sudah memenuhi semua target, jadi A dan B adalah ablation untuk bahan perbandingan sebelum/sesudah, bukan perbaikan darurat. Dasarnya cetak biru bagian 6 dan 9.

Run Perubahan Dasar di cetak biru Yang diuji

A d_dropout = 0.3 Cadangan 1 (bagian 6) D(fake) naik ke arah 0,2–0,4, L_G turun

B lr_d = 1e-4 (lr G tetap 2e-4) Bagian 9, “D terlalu kuat”, langkah 2 D belajar lebih lambat, D(fake) naik, L_G turun

Satu perubahan per run. run_ablation menolak dua perubahan sekaligus dan menolak CFG yang bukan baseline. Tes membuktikan config checkpoint A hanya berbeda di d_dropout, dan B hanya di lr_d.

Folder terpisah dan CFG selalu kembali. Tiap run punya folder runs/<nama>/, sehingga baseline tidak tertimpa. cfg_override mengembalikan CFG ke baseline walau training error atau dihentikan.

Pembanding sama. Evaluasi memakai clf dan ref yang sama dengan Bagian 7.

Bobot awal D identik. Bobot awal D sama dengan dan tanpa dropout (sudah dites), jadi perbandingannya adil di titik awal.

Dinilai sebelum dijalankan. Run dianggap lebih baik hanya bila akurasi dan keragaman tidak turun di luar kebisingan dan grid tidak lebih buruk. Loss yang membaik saja tidak cukup. Hasil “tidak berbeda dari baseline” adalah temuan yang sah.

3. Perbandingan

Sel perbandingan memuat tabel dengan kolom yang kamu minta, plus jumlah pemeriksaan WASPADA/PERHATIAN. Ada juga kurva keempat besaran dari tiga run dalam satu gambar dan grid epoch 50 berdampingan dengan grid acuan real. Semuanya dibaca dari log, CSV, dan PNG.

Selisih akurasi dibandingkan dengan 2× galat baku sampling. Variasi antar-seed tidak terukur dari satu run per konfigurasi, jadi selisih kecil jangan dibaca sebagai efek nyata.

Perkiraan waktu (T4)

Langkah Perkiraan

Satu run ±15–16 menit (50 epoch × ±18 dtk). Dropout kemungkinan sedikit lebih lambat, belum diukur

Evaluasi per run kurang dari 1 menit

Run A + B ±31–33 menit

Ruang Drive ±100 MB per run

Jalankan sel A dan sel B terpisah. Kalau sesi terputus, jalankan ulang sel yang sama: training lanjut dari checkpoint terbaru di folder run itu.

Urutan menjalankan

Tempel setelah Bagian 7, lalu jalankan sel 2 dan 3 (mesin eksperimen dan TES 8). Setelah itu jalankan sel run A, sel run B, sel perbandingan, dan terakhir sel keyakinan opsional. Sel A dan B memakai clf dan ref dari sel run Bagian 7, jadi sel itu harus sudah dijalankan di sesi yang sama.

Spectral norm (cadangan 2) sengaja belum dijalankan, karena baseline sudah memenuhi target dan aturan cetak biru adalah satu cadangan per kali. Bila nanti perlu, cukup tambah satu entri di EXPERIMENTS, misalnya {"d_spectral_norm": True}.

Kirim tabel perbandingan dan keluaran sel keyakinan setelah dijalankan, supaya saya bantu menyusun kesimpulan untuk presentasi.

Hasil run A/B dan permintaan kesimpulan presentasi

PROMPT / PESAN PENGGUNA

Run A dan B selesai dijalankan di Colab tanpa error, dan semua hasil sudah tersimpan di Drive dan GitHub. Tidak perlu membuat kode baru, saya hanya melaporkan hasil dan butuh penjelasan.

Tabel perbandingan epoch 50: baseline akurasi 92,33%, rasio keragaman min 0,92, L_D 0,718, L_G 2,550, D(real) 0,725, D(fake) 0,175; A (dropout 0,3) 93,14%, 0,85, 1,153, 1,382, 0,560, 0,339; B (lr_d 1e-4) 91,78%, 0,86, 0,902, 1,747, 0,637, 0,263. Selisih akurasi A +0,81 pp (di luar kebisingan sampling), B –0,55 pp (di dalam). Keyakinan classifier (sel 8.6): real 0,914 vs buatan 0,912, jadi dugaan "prototipikal" tidak didukung.

Tolong, untuk membantu presentasi:

1. Perbarui penjelasan kenapa akurasi gambar buatan (92,33%) sedikit di atas data asli (91,32%), mengingat dugaan prototipikal tidak didukung. Jujur bahwa penyebabnya belum terbukti.

2. Susun kesimpulan ablation untuk presentasi dengan bahasa sederhana dan jujur soal keterbatasan (satu seed per run, selisih kecil).

3. Beri saran checkpoint mana yang sebaiknya dipakai untuk demo generate() (baseline atau A) beserta alasannya, tanpa menulis kode.

JAWABAN AI

Ada koreksi dari saya dulu. Di sel 8.1 saya menulis dugaan “prototipikal” didukung oleh selisih kelas Shirt. Itu bukti lemah, dan sekarang tidak didukung oleh uji keyakinan. Paragraf itu perlu diganti dengan versi di bawah.

1. Akurasi buatan (92,33%) vs data asli (91,32%), versi diperbarui

Yang bisa dipegang

Selisihnya 1,0 poin persentase, kira-kira 2,6 kali galat baku sampling (perhitungan kasar saya). Jadi kemungkinan besar bukan kebetulan evaluasi, tetapi kecil.

Secara logika, kalau G meniru distribusi data dengan sempurna, classifier seharusnya memberi akurasi yang sama pada gambar buatan dan asli. Akurasi buatan yang lebih tinggi berarti, menurut classifier ini, kelas-kelas buatan lebih mudah dipisahkan daripada kelas-kelas asli.

Ini tidak berarti G lebih bagus dari data asli. Metriknya hanya mengukur label ditaati dan gambar mudah dikenali.

Yang sudah diuji

Dugaan “gambar buatan lebih prototipikal, jadi classifier lebih yakin” tidak didukung: keyakinan 0,914 (asli) vs 0,912 (buatan), praktis sama.

Satu pengamatan tambahan, bukan penjelasan: pada data asli keyakinan (0,914) sama dengan akurasi (0,913), sedangkan pada gambar buatan keyakinan (0,912) sedikit di bawah akurasi (0,923).

Uji ini hanya menyingkirkan satu bentuk dugaan. Dugaan lain tentang gambar yang “lebih bersih” belum tentu salah.

Kandidat yang tersisa (belum terbukti)

Celah generalisasi classifier. Classifier dilatih pada data train, dan G juga meniru data train. Gambar buatan mungkin lebih mirip distribusi train daripada test, sehingga akurasinya berada di antara akurasi test (91,3%) dan akurasi train. Ini bisa diperiksa murah: hitung akurasi classifier pada data train asli. Kalau hasilnya sekitar 92–94%, kandidat ini masuk akal. Kalau ini mau dijalankan, saya bisa buatkan satu barisnya.

G menonjolkan ciri pembeda kelas. D menerima label sebagai kanal tambahan, sehingga G mungkin melebih-lebihkan ciri khas tiap kelas. Kita belum punya uji untuk ini.

G kurang memuat gambar di daerah abu-abu antarkelas. Keragaman per piksel terlalu kasar untuk melihat hal ini. Rasio 0,92 hanya menunjukkan variasinya tidak hilang banyak.

Kalimat untuk slide: “Akurasi gambar buatan sedikit di atas data asli (92,3% vs 91,3%). Artinya label ditaati dengan baik, bukan bahwa gambar buatan lebih bagus dari yang asli. Penyebab pastinya belum kami buktikan. Dugaan ‘gambar lebih prototipikal’ sudah kami uji dan tidak didukung.”

2. Kesimpulan ablation

Prediksi Hasil A (dropout 0,3) Hasil B (lr D 1e-4)

```
D(fake) naik■ya■0,175 → 0,339 ✓■0,175 → 0,263 ✓
```

L_G turun ya 2,55 → 1,38 ✓ 2,55 → 1,75 ✓

Akurasi berubah kecil +0,81 pp (±2,2 galat baku, batas) –0,55 pp (±1,4 galat baku, dalam kebisingan)

Rasio keragaman min berubah kecil 0,92 → 0,85 0,92 → 0,86

Kesimpulan yang boleh disampaikan

Tafsir loss terdukung. Dua cara berbeda untuk memperlemah D sama-sama menggeser D(fake) naik dan L_G turun. Ini mendukung tafsir bahwa kenaikan L_G di baseline berkaitan dengan D yang makin kuat. Ini dukungan, bukan bukti pasti.

Loss yang lebih seimbang tidak otomatis membuat sampel lebih baik. Run B masuk pita sehat di kedua sisi (D(real) 0,637 dan D(fake) 0,263), tetapi akurasinya tidak lebih tinggi. Ini sejalan dengan pesan cetak biru: jangan menilai dari loss saja.

Tidak ada bukti bahwa A atau B lebih baik secara menyeluruh. Kriteria yang kita tetapkan sebelum run adalah akurasi dan keragaman tidak turun di luar kebisingan. A menang tipis di akurasi tetapi rasio keragamannya turun 0,07. B tidak menang apa-apa. Baseline sudah cukup, dan itu temuan yang sah.

Keragaman sedikit turun di kedua run, tetap jauh di atas ambang 0,5. Arah turunnya sama di A dan B. Karena hanya satu seed, belum jelas apakah itu efek nyata.

Keterbatasan yang perlu disebut jujur

Satu seed per run. Variasi antar-seed tidak terukur. Galat baku yang kita pakai hanya kebisingan pengambilan sampel gambar. Dari log baseline, akurasi antar-checkpoint epoch 20–50 hanya bergerak ±0,2 poin, tetapi itu variasi dalam satu run, bukan antar-seed.

Selisih kecil. Selisih A sekitar 2,2 galat baku, dan ada banyak metrik yang dibandingkan. Perlakukan sebagai petunjuk, bukan temuan pasti.

Loss A tidak sebanding persis dengan baseline. D di run A dihitung dengan dropout aktif, sehingga L_D 1,153 dan D(real) 0,560 ikut terpengaruh derau dropout.

Hanya satu nilai per perubahan (0,3 dan 1e-4), jadi bukan kurva tuning.

Metrik berbasis classifier hanya mengukur ketaatan label. Penilaian kualitas visual tetap lewat grid, dan itu keputusan matamu. Saya belum melihat grid-nya, jadi kesimpulan visual harus dari kamu.

Bila ada waktu, mengulang baseline dan A dengan 2–3 seed (±15 menit per run) akan menjawab pertanyaan “efek nyata atau kebetulan”. Itu bukan syarat untuk presentasi.

Kalimat untuk slide: “Memperlemah D membuat loss tampak lebih seimbang, tetapi kualitas sampel tidak jelas lebih baik. Dengan satu seed per run dan selisih yang kecil, kesimpulan kami: baseline sudah memadai, dan loss yang seimbang bukan jaminan gambar yang lebih baik.”

3. Checkpoint untuk demo generate()

Saran: baseline, epoch 50 (ckpt_epoch050.pt di folder utama).

Alasannya:

Semua target sudah terpenuhi, dan akurasi baseline datar sejak epoch 20. Tidak ada alasan metrik untuk memilih epoch lebih awal.

Demo generate() justru memamerkan variasi di dalam satu kelas. Baseline punya keragaman tertinggi (rasio min 0,92 vs 0,85 di A).

Keunggulan A hanya +0,81 poin akurasi yang masih batas. Itu tidak sebanding dengan penurunan keragaman dan dengan menjadikan eksperimen sebagai model utama.

Baseline adalah run acuan untuk semua grafik lain, jadi ceritanya konsisten: satu model dari data sampai demo.

Catatan praktis:

Pakai A sebagai pembanding, bukan model utama. Tampilkan A epoch 50 di slide ablation, berdampingan dengan baseline. Checkpoint A bisa dimuat generate() tanpa perubahan karena arsitektur G sama. Itu sudah terbukti saat evaluate_all memuat checkpoint A di Colab.

Satu pengecualian. Kalau menurut matamu grid A epoch 50 jelas lebih tajam, kamu boleh memilih A, asalkan kamu menyebutkan alasan visualnya. Metrik kita tidak memisahkan keduanya dengan tegas.

Pakai seed tetap supaya demo identik setiap kali. Tunjukkan dua hal: satu noise dengan label berbeda (efek label), dan satu label dengan noise berbeda (variasi). Contoh dari cetak biru: kelas 7 (Sneaker) dengan n=16.

Simpan PNG-nya lebih dulu sebagai cadangan, kalau GPU atau Colab bermasalah saat presentasi.